

Algorithms for Data Science

Lecture 6: Handling Imbalanced Dataset and Building ML model





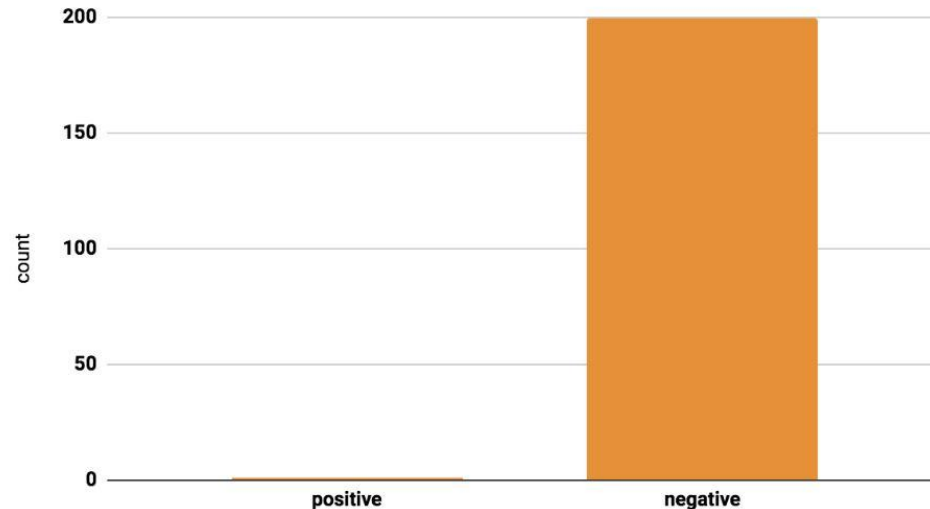
Table of Contents

- Handling Imbalanced Dataset
- SMOTE
- Machine Learning
- KNN
- Weighted KNN



Handling Imbalanced Dataset

- Handling imbalanced datasets is a common challenge in machine learning, where one class significantly outnumbers the other(s). This imbalance can lead to biased models that perform well on the majority class but poorly on the minority class.



Handling Imbalanced Dataset

- Below are strategies to handle imbalanced datasets:
 - ✓ Resampling Techniques
 - ✓ Algorithmic Approaches
 - ✓ Data-Level Approaches
- All approaches are complementary and can be used together to improve model performance on imbalanced datasets.



Resampling Techniques

- Resampling involves modifying the dataset to balance the class distribution. This can be done by oversampling the minority class, or undersampling the majority class, or combine oversampling and undersampling to balance the dataset.

a. Oversampling the Minority Class

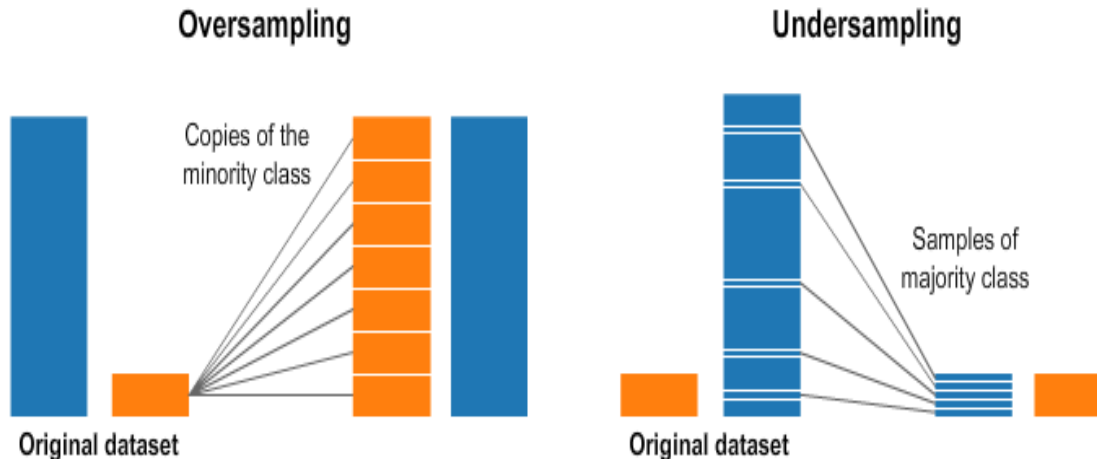
- **What it does:** Increase the number of instances in the minority class by duplicating existing samples or generating synthetic samples.
- **Methods:**
 - **Random Oversampling:** Randomly duplicate samples from the minority class.
 - **SMOTE (Synthetic Minority Oversampling Technique):** Create synthetic samples by interpolating between existing minority class samples.



Resampling Techniques

b. Undersampling the Majority Class

- **What it does:** Reduce the number of instances in the majority class by randomly removing samples.
- **Methods:**
 - **Random Undersampling:** Randomly remove samples from the majority class.



Resampling Techniques- SMOTE

How SMOTE Works

1. Identify Minority Class Samples:

- Find all samples belonging to the minority class.

2. Find k Nearest Neighbors:

- For each minority class sample, find its k nearest neighbors (also from the minority class).

3. Generate Synthetic Samples:

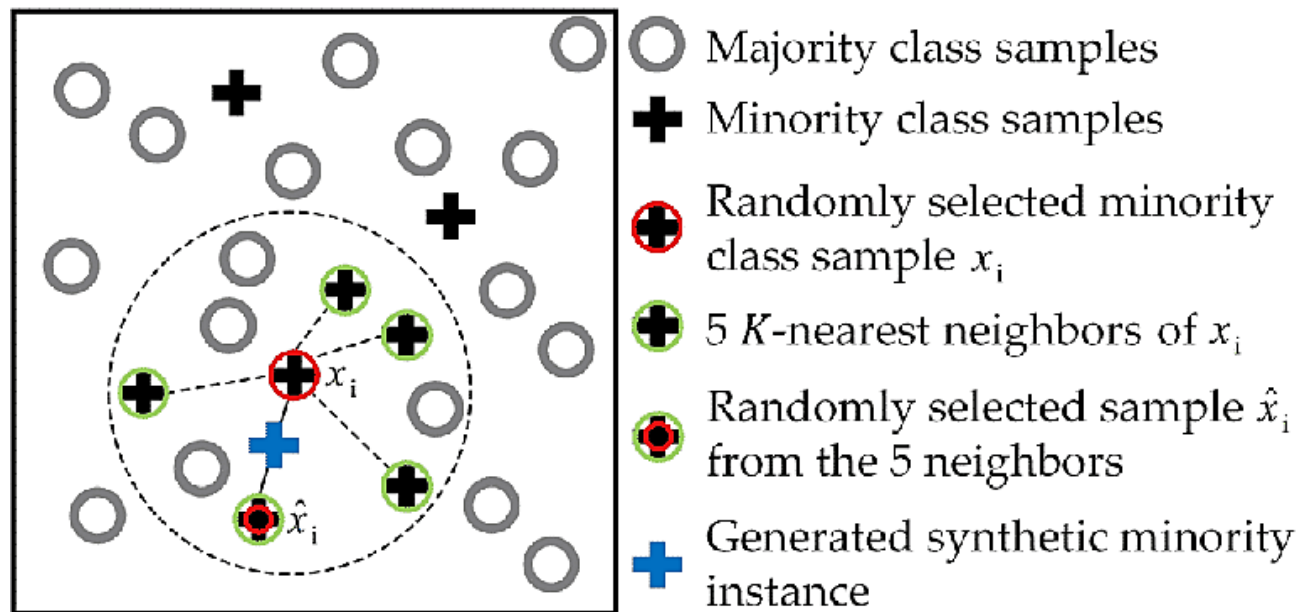
- Randomly select one of the k nearest neighbors.
- Create a synthetic sample along the line segment joining the original sample and its selected neighbor.

4. Repeat:

- Repeat the process until the desired number of synthetic samples is generated.



Resampling Techniques- SMOTE



Resampling Techniques- SMOTE

Mathematical Explanation

For a minority class sample x_i , SMOTE generates a synthetic sample x_{new} as follows:

$$x_{\text{new}} = x_i + \lambda \cdot (x_{zi} - x_i)$$

Where:

- x_i : Original minority class sample.
- x_{zi} : One of the k nearest neighbors of x_i .
- λ : A random number between 0 and 1, controlling the interpolation.



Resampling Techniques- SMOTE (Example)

Consider the following imbalanced dataset with **2 features** (F_1, F_2) and a **binary target** (Y):

F_1	F_2	Y
1	2	0
2	3	0
3	4	0
4	5	0
5	6	0
10	11	1
12	13	1
14	15	1

- **Majority Class** ($Y = 0$): 5 samples.
- **Minority Class** ($Y = 1$): 3 samples.



Resampling Techniques- SMOTE (Example)

Step 1: Identify Minority Class Samples

The minority class has three samples:

$$x_1 = (10, 11), \quad x_2 = (12, 13), \quad x_3 = (14, 15)$$



Resampling Techniques- SMOTE (Example)

Step 2: Find k Nearest Neighbors

For each minority class sample, find its k nearest neighbors (also from the minority class). Let's use $k = 2$ for this example.

For $x_1 = (10, 11)$:

- Nearest neighbors: $x_2 = (12, 13)$ and $x_3 = (14, 15)$.

For $x_2 = (12, 13)$:

- Nearest neighbors: $x_1 = (10, 11)$ and $x_3 = (14, 15)$.

For $x_3 = (14, 15)$:

- Nearest neighbors: $x_1 = (10, 11)$ and $x_2 = (12, 13)$.



Resampling Techniques- SMOTE (Example)

Step 3: Generate Synthetic Samples

For each minority class sample, generate synthetic samples by interpolating with its nearest neighbors.

For $x_1 = (10, 11)$:

- Nearest neighbors: $x_2 = (12, 13)$ and $x_3 = (14, 15)$.
- Generate synthetic samples:
 - With x_2 (let $\lambda = 0.5$):

$$x_{\text{new}} = (10, 11) + 0.5 \cdot ((12, 13) - (10, 11)) = (10, 11) + 0.5 \cdot (2, 2) = (11, 12)$$

- With x_3 (let $\lambda = 0.5$):

$$x_{\text{new}} = (10, 11) + 0.5 \cdot ((14, 15) - (10, 11)) = (10, 11) + 0.5 \cdot (4, 4) = (12, 13)$$



Resampling Techniques- SMOTE (Example)

Step 3: Generate Synthetic Samples

For $x_2 = (12, 13)$:

- Nearest neighbors: $x_1 = (10, 11)$ and $x_3 = (14, 15)$.

- Generate synthetic samples:

- With x_1 (let $\lambda = 0.5$):

$$x_{\text{new}} = (12, 13) + 0.5 \cdot ((10, 11) - (12, 13)) = (12, 13) + 0.5 \cdot (-2, -2) = (11, 12)$$

- With x_3 (let $\lambda = 0.5$):

$$x_{\text{new}} = (12, 13) + 0.5 \cdot ((14, 15) - (12, 13)) = (12, 13) + 0.5 \cdot (2, 2) = (13, 14)$$



Resampling Techniques- SMOTE (Example)

Step 3: Generate Synthetic Samples

For $x_3 = (14, 15)$:

- Nearest neighbors: $x_1 = (10, 11)$ and $x_2 = (12, 13)$.
- Generate synthetic samples:
 - With x_1 (let $\lambda = 0.5$):

$$x_{\text{new}} = (14, 15) + 0.5 \cdot ((10, 11) - (14, 15)) = (14, 15) + 0.5 \cdot (-4, -4) = (12, 13)$$

- With x_2 (let $\lambda = 0.5$):

$$x_{\text{new}} = (14, 15) + 0.5 \cdot ((12, 13) - (14, 15)) = (14, 15) + 0.5 \cdot (-2, -2) = (13, 14)$$



Resampling Techniques- SMOTE (Example)

After applying SMOTE, the updated dataset is:

F_1	F_2	Y
1	2	0
2	3	0
3	4	0
4	5	0
5	6	0
10	11	1
11	12	1
12	13	1
13	14	1
14	15	1

- **Majority Class** ($Y = 0$): 5 samples.
- **Minority Class** ($Y = 1$): 5 samples.



Handling Imbalanced Dataset- Algorithmic Approaches

- Modify the learning algorithm to account for class imbalance.
- **Class Weights is the most common technique in this approach.**
 - **What it does:** Assign higher weights to the minority class during model training.
 - **How it works:**
 - In algorithms like logistic regression, SVM, or decision trees, you can specify `class_weight` to penalize misclassification of the minority class more heavily.
 - Example: `class_weight='balanced'` in scikit-learn.



Handling Imbalanced Dataset- Data-Level Approaches

- Data-level approaches focus on modifying the dataset or the way data is used to address class imbalance. This can be done by collecting more data or apply data augmentation techniques.

a. Collect More Data

- **What it does:** Collect additional samples for the minority class.
- **Pros:**
 - Improves model performance without synthetic data.
- **Cons:**
 - Often impractical or expensive.



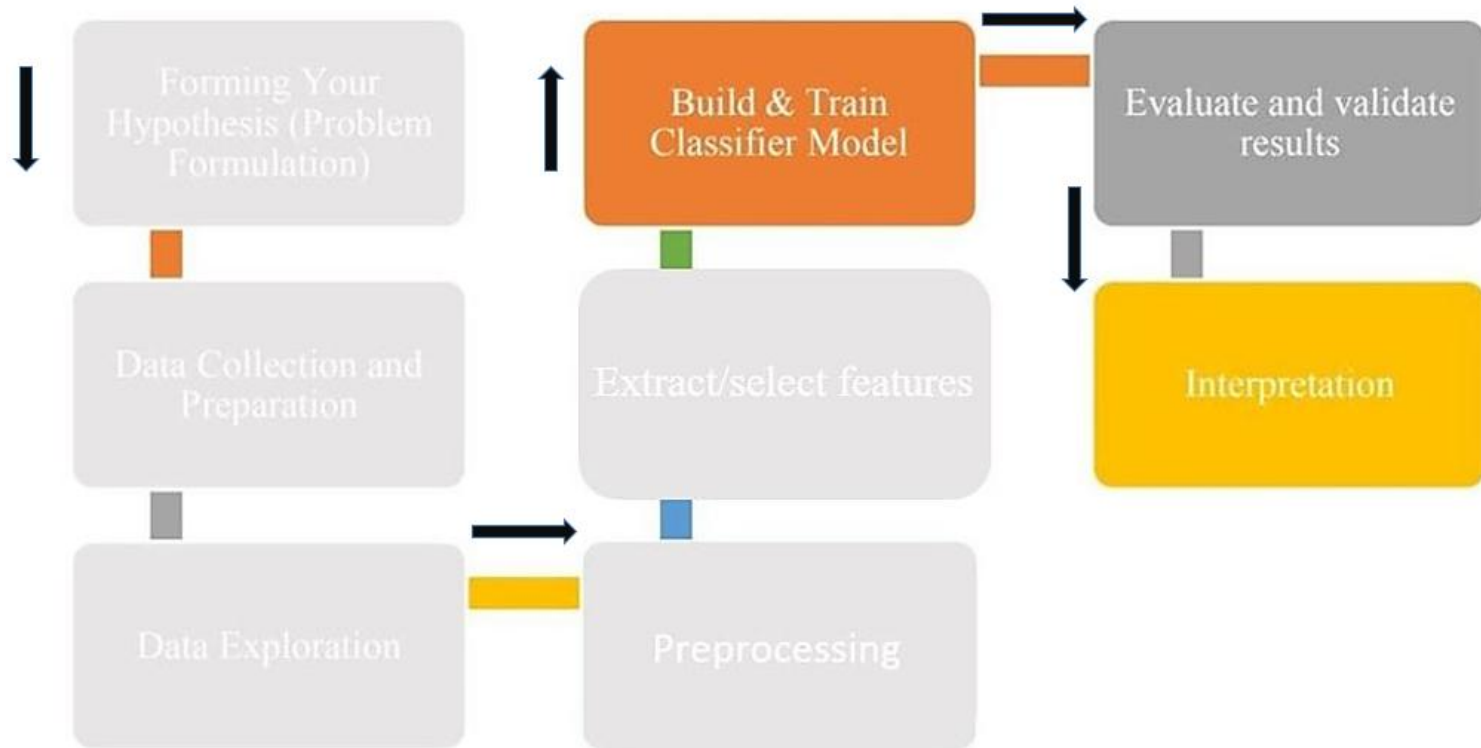
Handling Imbalanced Dataset- Data-Level Approaches

b. Data Augmentation

- **What it does:** Generate new samples for the minority class using transformations (e.g., rotation, flipping for images).
- **Pros:**
 - Increases diversity of the minority class.
- **Cons:**
 - Limited to specific types of data (e.g., images, text).

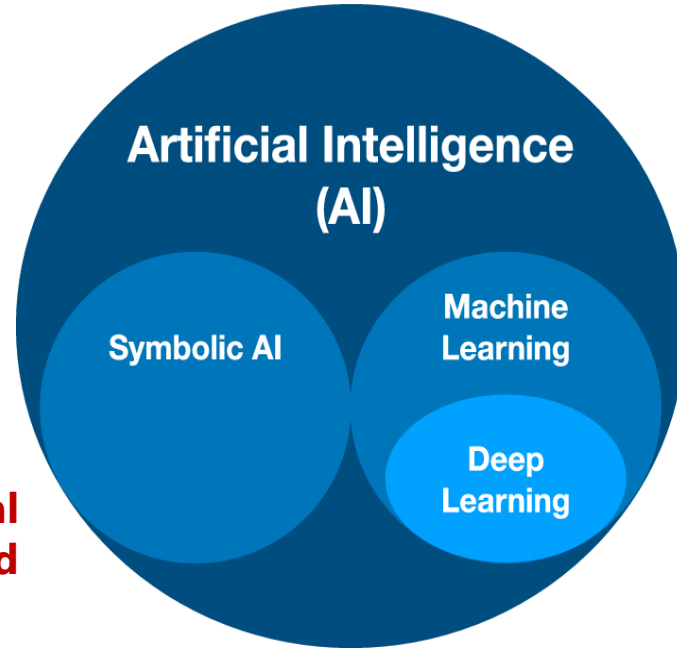


The Data Science Pipeline



Big Umbrella (1/3)

Graphical representation of Artificial Intelligence, Machine Learning, and Deep Learning



Big Umbrella (2/3)

- AI aiming to create systems that perform tasks requiring human-like intelligence.
- Symbolic AI (Traditional AI): A classical approach where rules and logic are hand-coded by humans (e.g., expert systems).
- To contrast with Machine learning (ML): Symbolic AI lacks learning capability, whereas ML systems improve autonomously from data.

Traditional Programming



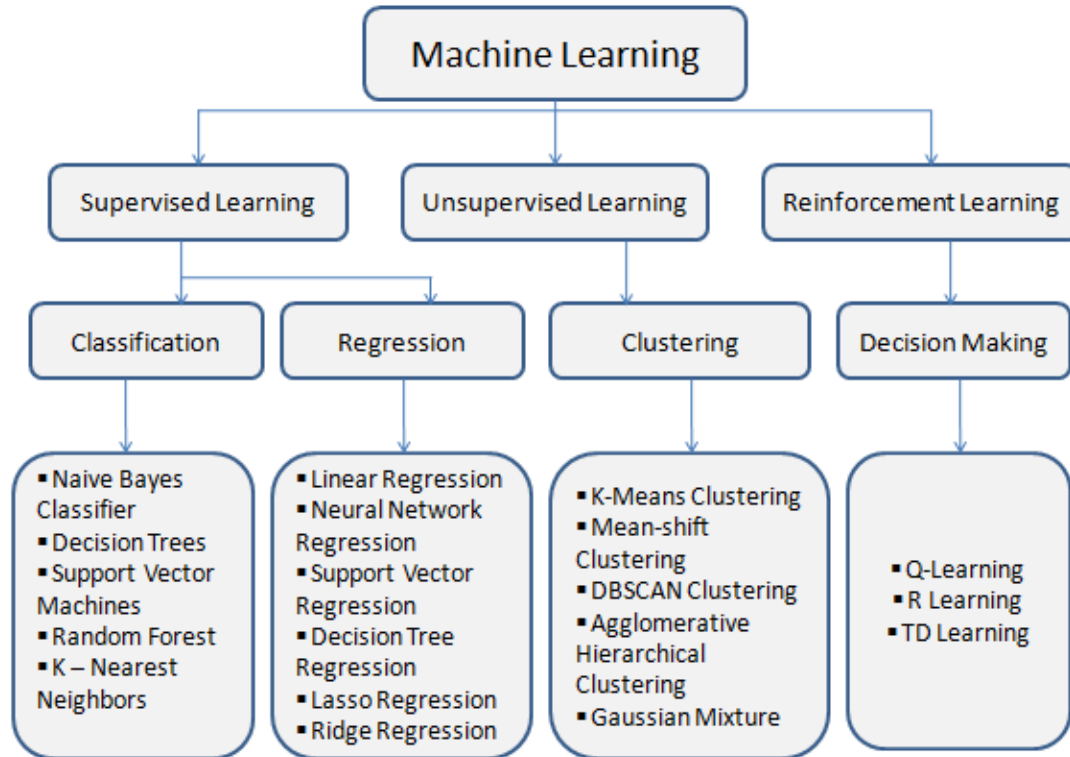
Big Umbrella (3/3)

- Machine learning (ML) is a subset of AI where systems **learn patterns from data** instead of relying on explicit rules **(Data-Driven AI)**.
- ML algorithms adapt through experience (e.g., spam filters, recommendation systems).
- Deep Learning (DL): A subset of ML that uses multi-layered neural networks to **learn complex patterns** (e.g., image recognition, ChatGPT).

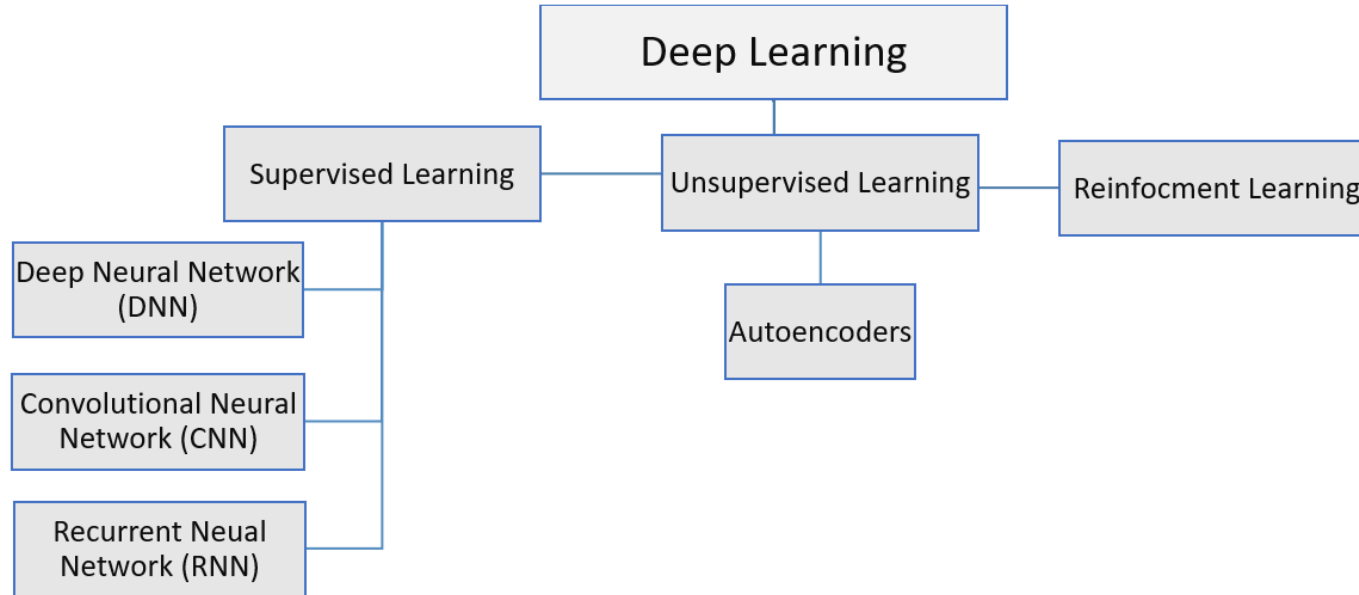
Machine Learning



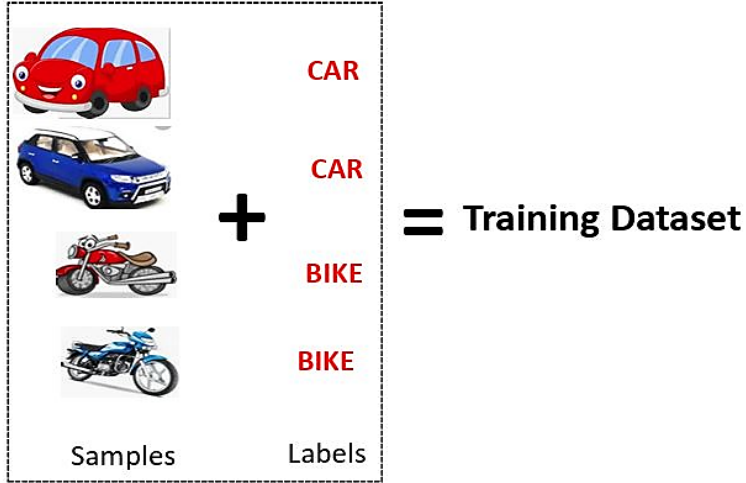
Taxonomy of Machine Learning Algorithms



Taxonomy of Deep Learning Algorithms



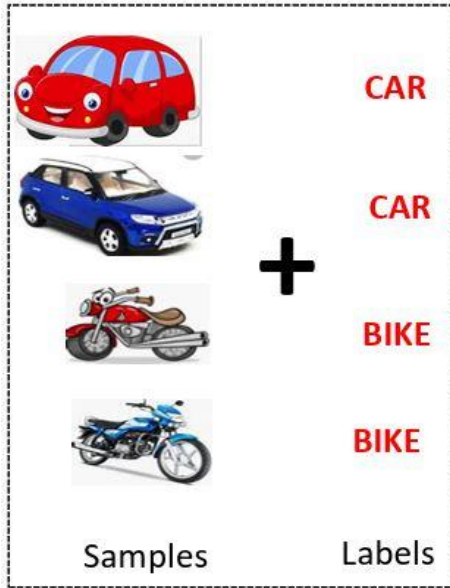
Supervised Learning



In supervised learning, we need some thing called a Labelled Training Dataset



Supervised Learning

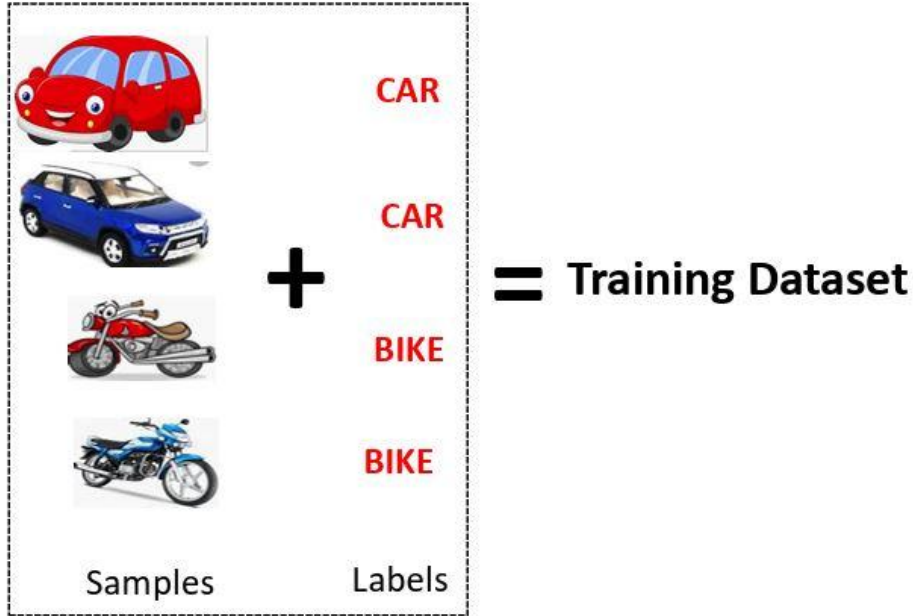


= Training Dataset

$$f(\text{dataset}, \text{new sample}) =$$

Given a labelled dataset, the task is to devise a function which takes the dataset, and a new sample, and produces an output value.

Supervised Learning

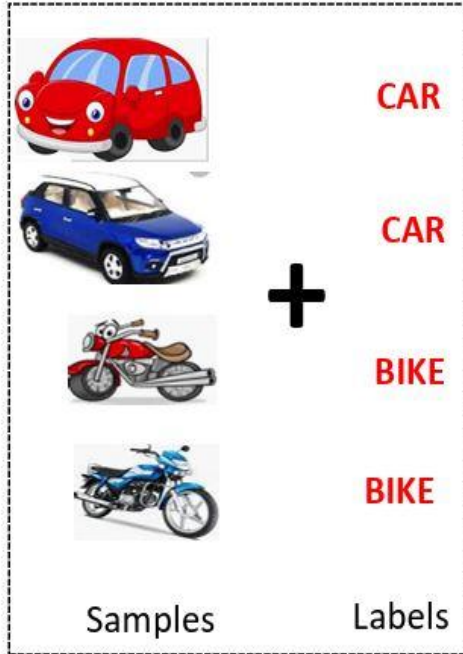


Classification

$$f(\text{blue cylinder}, \text{yellow sports car}) = \text{CAR}$$

If the possible output values of the function are predefined and discrete/categorical, it is called Classification

Supervised Learning



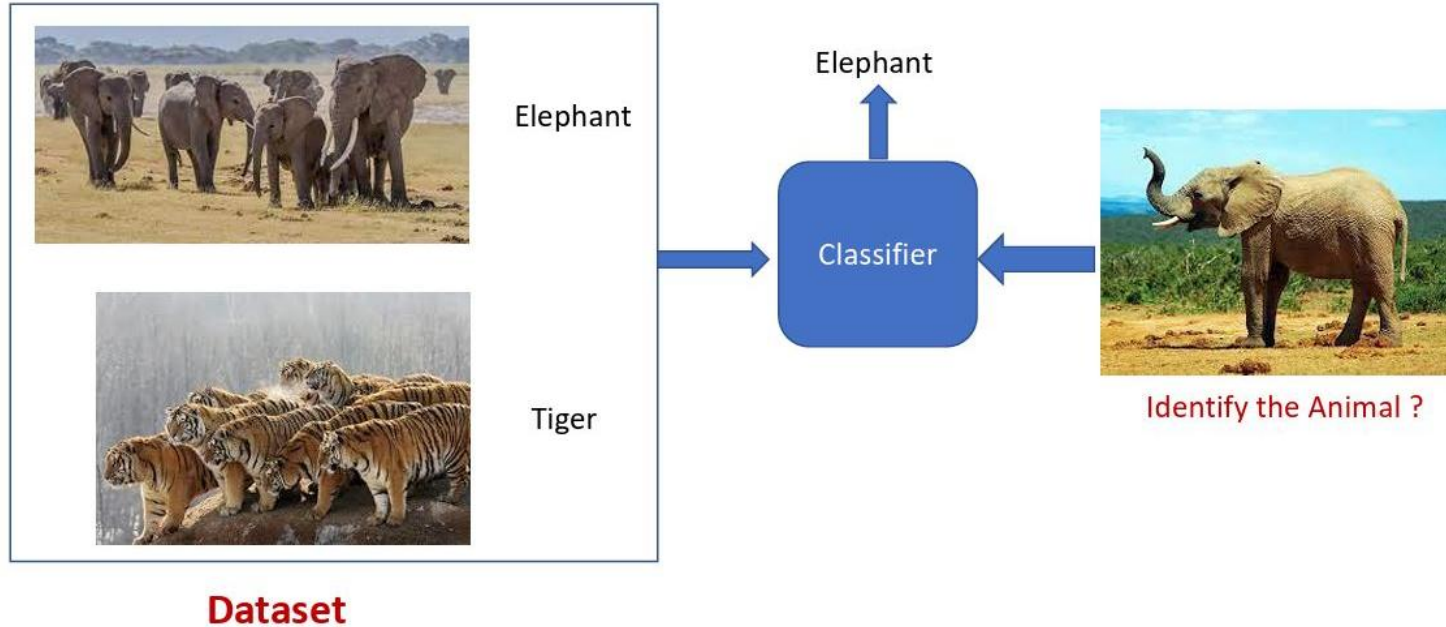
= Training Dataset

Classification

$$f(\text{blue cylinder}, \text{yellow bus}) = \text{CAR}$$

Predefined classes means, it will produce output only from the labels defined in the dataset. For example, even if we input a bus, it will produce either CAR or BIKE

Supervised Learning- Classification



There is a wide variety of machine learning (ML) and deep learning (DL) algorithms available for classification tasks.

Supervised Learning- Classification

- There are three main types of classification, depending on the nature of the problem (dataset): **Binary Classification**, **Multiclass Classification**, and **Multilabel Classification**.
- **Binary Classification**: Only two possible classes (e.g., Yes/No, Spam/Not Spam, Fraud/Not Fraud).
- **Multiclass Classification**: More than two classes, and each instance is assigned to one class. Example: Classifying images into cats, dogs, or birds.
- **Multilabel Classification**: A single instance can belong to multiple classes simultaneously. Example: A movie can be tagged as both "Action" and "Comedy."



Supervised Learning- Regression



Dataset

Regression

$$f(\text{cylinder}, \text{house}) = 20500.50$$

If the possible output values of the function are continuous real values, then it is called Regression



Supervised Learning

- Both classification and regression problems fall under supervised learning.
- Supervised Learning means the model is trained on labeled data, where each input example is paired with a known output (the "ground truth").
- Classification predicts discrete class labels (e.g., spam/not spam, dog/cat).
Regression predicts continuous numerical values (e.g., house prices, temperature).
- The model's decisions are directly influenced by the labeled data (experience).
- The learning algorithm adjusts its parameters to minimize the difference between its predictions and the true labels/values.



Unsupervised Learning



~~CAR~~



~~CAR~~



~~BIKE~~



~~BIKE~~

Dataset

In the unsupervised learning, we do not need to know the labels or Ground truth values.



Unsupervised Learning



Clustering

Dataset

The task is to identify the patterns like group the similar objects together.



Unsupervised Learning- More Examples



Dataset



The task is to identify the patterns like group the similar objects together.

Unsupervised Learning- More Examples

More Example Unsupervised Learning



Customers who viewed this item also viewed

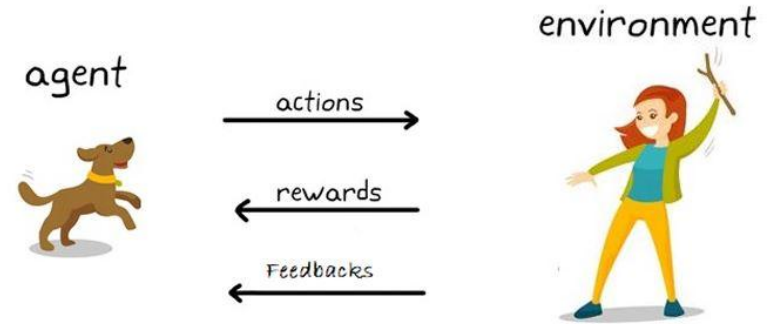


The task is to identify the patterns like group the similar objects together.



Reinforcement Learning

- Reinforcement Learning (RL) is a type of machine learning where an agent learns to make decisions by interacting with an environment to maximize a reward signal.
- Unlike supervised learning (which relies on labeled data) or unsupervised learning (which finds patterns in unlabeled data), RL is driven by trial and error, with feedback in the form of rewards or penalties.



Reinforcement Learning

- Reinforcement Learning (RL) can be **divided into Online Reinforcement Learning and Offline Reinforcement Learning** based on how the agent interacts with the environment and learns from data.
- Online Reinforcement Learning: The agent interacts with the environment in real-time, collecting new experiences and learning from them immediately. It continuously explores (tries new actions) and exploits (uses learned knowledge).
- Offline Reinforcement Learning (Batch RL): The agent learns from a fixed dataset of past experiences (collected from other policies or human demonstrations) without interacting with the environment.



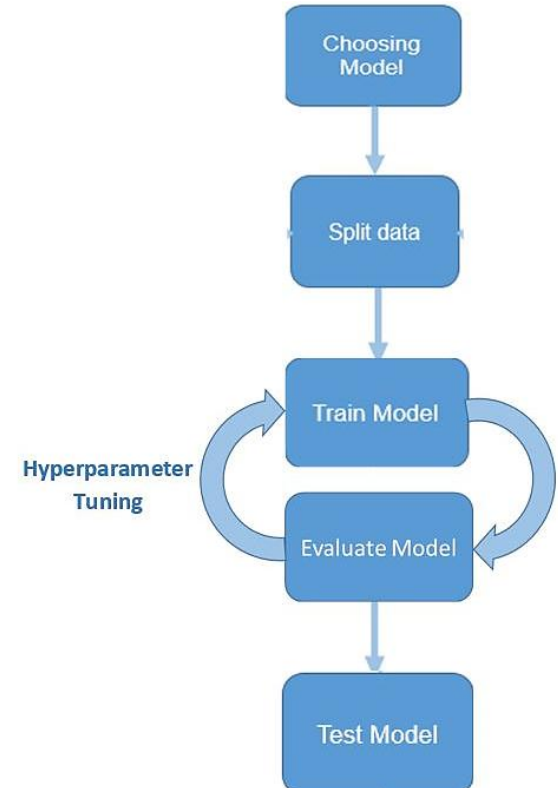
Machine Learning Workflow

1. Choosing a Model

- **Goal:** Select an algorithm suited to the problem type and data.
- **Examples:**
 - **Classification:** Logistic Regression, Random Forest, SVM.
 - **Regression:** Linear Regression, Gradient Boosting.
 - **Clustering:** K-Means (if no labels are available).

2. Splitting the Data

- **Goal:** Divide data into subsets to train and test the model fairly.
- **Common Splits:**
 - **Train (70-80%):** Used to train the model.
 - **Test (20-30%):** Used to evaluate final performance.
 - **Validation (optional):** Used for hyperparameter tuning.



Machine Learning Workflow

3. Train the Model

- **Goal:** Fit the model to the training data.

4. Evaluate Model

Assess the model's performance on the validation set using metrics like accuracy, precision, or RMSE to identify potential improvements.

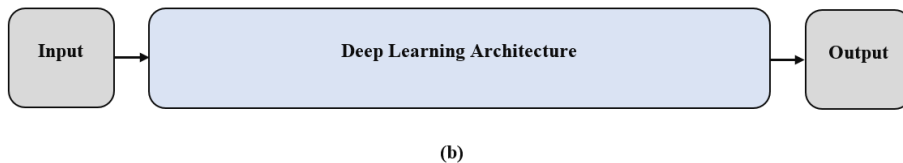
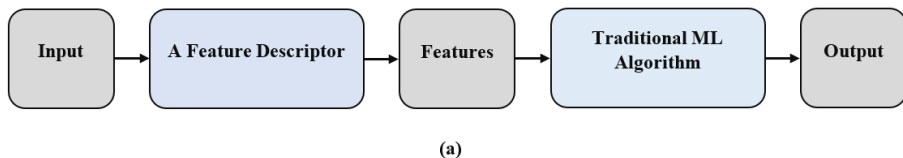
5. Test Model

Validate the final model on the unseen test set to estimate its real-world performance and generalization ability.



Machine learning vs. Deep learning

- Machine learning algorithms tend to focus on learning only one or two layers of representations of the data. Therefore, traditional machine learning often needs other methods to help get the best results.
- Deep learning is a new take on learning representations from data that puts an emphasis on learning successive layers of increasingly meaningful representations.
- In the context of image classification, traditional machine learning needs a feature extraction tool, while deep learning can extract features on its own.

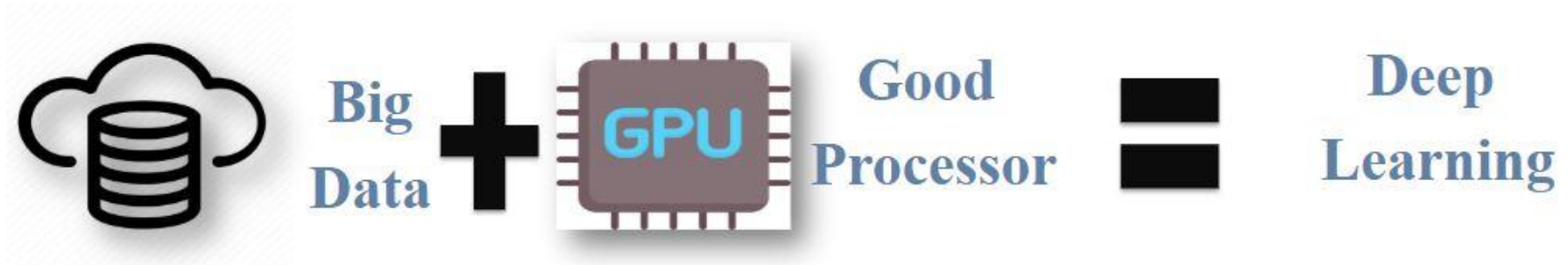


Machine learning vs. Deep learning

Factors	Deep Learning	Machine Learning
Data Requirement	Requires Large Data	Can train on less data
Accuracy	Gives High Accuracy	Gives Less Accuracy
Training Time	Takes Longer to Train	Takes Less Time to Train
Hardware Dependency	Requires GPU to Train Probably	Trains on CPU



When are deep learning algorithms used?



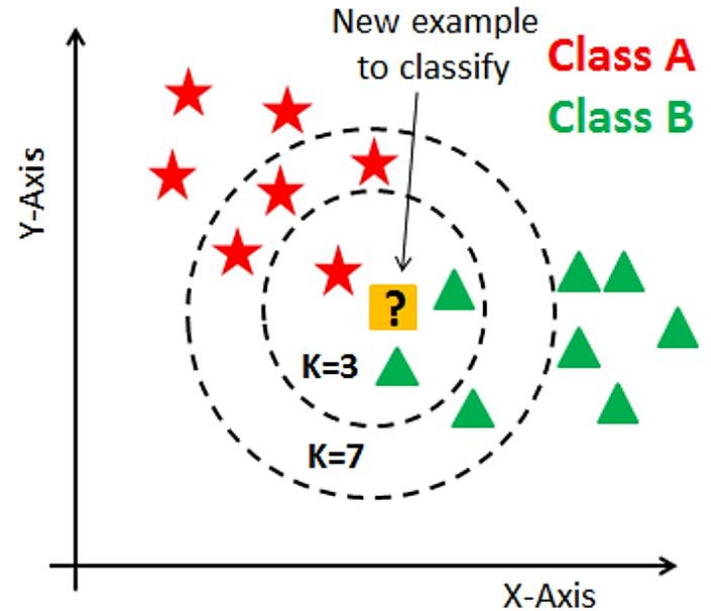
Let's start with the first ML algorithm:

K-Nearest Neighbors (KNN) Algorithm



KNN Algorithm

- The k-Nearest Neighbors (k-NN) algorithm is a supervised machine learning method used for classification and regression.
- It works by finding the k closest training examples to a new data point and making predictions based on their labels (for classification) or values (for regression).



KNN Algorithm

- The k-Nearest Neighbors (k-NN) algorithm is often referred to as a "lazy learner" (or instance-based learner) because of its unique approach to learning. Unlike eager learners (e.g., Neural Networks), which construct a generalized model during training, k-NN does not learn any explicit model beforehand. Instead, it postpones computation until prediction time.
- It does not compute any model during training. Simply KNN stores the entire training dataset as-is and all computation happens at prediction time.



KNN Algorithm: Steps

1. **Choose the value of k** (number of neighbors to consider).
2. **Calculate the distance** between the new data point and all training points.
3. **Select the k nearest neighbors** based on the smallest distances.
4. **For Classification:** Assign the majority class among neighbors.
For Regression: Assign the average value of neighbors.
5. **Output the prediction.**



KNN Algorithm: Distance Metrics

The choice of distance metric affects the performance of k-NN. Common metrics include:

1. Euclidean Distance (Most Common)

$$d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2}$$

- Measures the straight-line distance between two points.
- **Example:**
 - Point A: (2, 3)
 - Point B: (5, 7)
 - Distance = $\sqrt{(5 - 2)^2 + (7 - 3)^2} = \sqrt{9 + 16} = 5$



KNN Algorithm: Distance Metrics

2. Manhattan Distance

$$d(x, y) = |x_1 - y_1| + |x_2 - y_2| + \cdots + |x_n - y_n|$$

- Measures distance along grid-like paths (like city blocks).
- **Example:**
 - Point A: (2, 3)
 - Point B: (5, 7)
 - Distance = $|5 - 2| + |7 - 3| = 3 + 4 = 7$



KNN Algorithm: Classification problem example

Problem: Predict whether a fruit is an **Apple** or **Orange** based on **Weight (grams)** and **Smoothness (1-10 scale)**.

Weight (g)	Smoothness	Fruit
150	8	Apple
170	7	Apple
140	6	Orange
130	5	Orange
160	9	Apple

New Data Point:

- Weight = **155g**, Smoothness = **7.5**
- Let's choose **k=3** (3 nearest neighbors).



KNN Algorithm: Classification problem example

Step 1: Calculate Distances (Euclidean Distance)

- Distance to (150,8) = $\sqrt{[(155-150)^2 + (7.5-8)^2]} \approx \mathbf{5.02}$
- Distance to (170,7) = $\sqrt{[(155-170)^2 + (7.5-7)^2]} \approx \mathbf{15.03}$
- Distance to (140,6) = $\sqrt{[(155-140)^2 + (7.5-6)^2]} \approx \mathbf{15.13}$
- Distance to (130,5) = $\sqrt{[(155-130)^2 + (7.5-5)^2]} \approx \mathbf{25.24}$
- Distance to (160,9) = $\sqrt{[(155-160)^2 + (7.5-9)^2]} \approx \mathbf{5.59}$

Step 2: Pick 3 Nearest Neighbors

1. (150,8) → **Apple** (distance = 5.02)
2. (160,9) → **Apple** (distance = 5.59)
3. (170,7) → **Apple** (distance = 15.03)

Step 3: Majority Vote

- All 3 neighbors are **Apple** → Predicted class = **Apple** ✓



KNN Algorithm: Regression problem example

Problem: Predict the **price (\$)** of a house based on **Size (sq. ft.)** and **Bedrooms**.

Size (sq. ft.)	Bedrooms	Price (\$)
1500	2	300,000
1600	3	320,000
1400	2	290,000
1700	3	330,000
1550	2	310,000

New Data Point:

- Size = **1520 sq. ft.**, Bedrooms = **2**
- Let's choose **k=3**.



KNN Algorithm: Regression problem example

Step 1: Calculate Distances

- Distance to (1500,2) = $\sqrt{[(1520-1500)^2 + (2-2)^2]} = \mathbf{20}$
- Distance to (1600,3) = $\sqrt{[(1520-1600)^2 + (2-3)^2]} \approx \mathbf{80.01}$
- Distance to (1400,2) = $\sqrt{[(1520-1400)^2 + (2-2)^2]} = \mathbf{120}$
- Distance to (1700,3) = $\sqrt{[(1520-1700)^2 + (2-3)^2]} \approx \mathbf{180.01}$
- Distance to (1550,2) = $\sqrt{[(1520-1550)^2 + (2-2)^2]} = \mathbf{30}$

Step 2: Pick 3 Nearest Neighbors

1. (1500,2) → **\$300,000** (distance = 20)
2. (1550,2) → **\$310,000** (distance = 30)
3. (1600,3) → **\$320,000** (distance = 80.01)

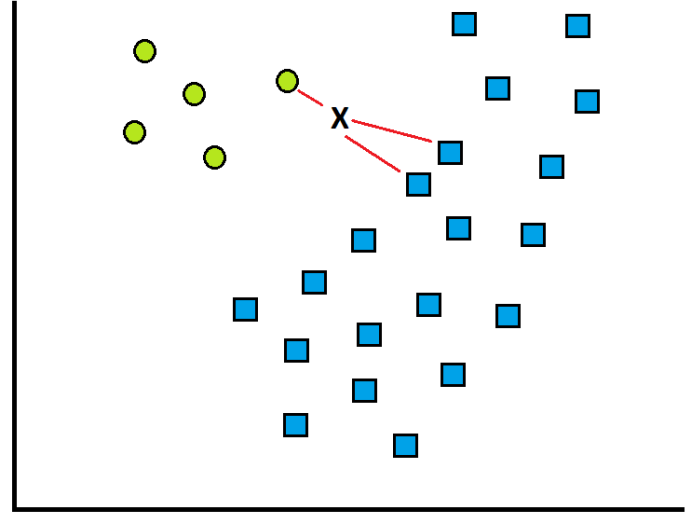
Step 3: Average the Prices

- Predicted Price = $(300,000 + 310,000 + 320,000) / 3 = \mathbf{\$310,000}$ ✓



Weighted KNN Algorithm

- Weighted k-NN is an improved version of the standard k-NN algorithm where closer neighbors contribute more to the prediction than farther ones.
- Instead of treating all neighbors equally, we assign weights based on their distance from the test point.



Weighted KNN Algorithm: Steps

1. **Find k nearest neighbors** (same as standard k -NN).
2. **Assign weights** to each neighbor (usually inverse distance or Gaussian weights).
3. **Make predictions:**
 - **Classification:** Weighted voting (closer neighbors have higher influence).
 - **Regression:** Weighted average (closer neighbors contribute more).

Common Weighting Schemes

Weight Type	Formula	Behavior
Inverse Distance	$w_i = \frac{1}{d_i}$	Closer points get much higher weights
Inverse Squared Distance	$w_i = \frac{1}{d_i^2}$	Stronger penalty for far points
Gaussian (Exponential) Weight	$w_i = e^{-d_i}$	Smooth weight decay



Classification Example

Problem: Should We Water the Plant?

Features: Soil Moisture (%), Sunlight (hours/day)

Classes: Water (**W**), Don't Water (**N**)

Training Data (5 Samples)

Moisture	Sunlight	Action	
20	2	W	(Dry, low sun → Water)
30	5	N	(Moist, high sun → Don't water)
25	3	W	(Borderline case)
22	4	N	(Slightly dry, moderate sun)
28	6	N	(Moist, high sun)

New Observation

- **Moisture = 24%, Sunlight = 3.5 hrs**

Let's apply KNN and
weighted KNN to discover
the difference



Classification Example

Step 1: Calculate Distances (Euclidean)

For the new point (24, 3.5), compute distances to all 5 training points:

1. Distance to (20, 2):

$$\sqrt{(24 - 20)^2 + (3.5 - 2)^2} = \sqrt{16 + 2.25} = \sqrt{18.25} \approx 4.27$$

2. Distance to (30, 5):

$$\sqrt{(24 - 30)^2 + (3.5 - 5)^2} = \sqrt{36 + 2.25} = \sqrt{38.25} \approx 6.18$$

3. Distance to (25, 3):

$$\sqrt{(24 - 25)^2 + (3.5 - 3)^2} = \sqrt{1 + 0.25} = \sqrt{1.25} \approx 1.12$$

4. Distance to (22, 4):

$$\sqrt{(24 - 22)^2 + (3.5 - 4)^2} = \sqrt{4 + 0.25} = \sqrt{4.25} \approx 2.06$$

5. Distance to (28, 6):

$$\sqrt{(24 - 28)^2 + (3.5 - 6)^2} = \sqrt{16 + 6.25} = \sqrt{22.25} \approx 4.72$$

Sorted Distances (Nearest to Farthest):

1. (25, 3): 1.12 (W)

2. (22, 4): 2.06 (N)

3. (20, 2): 4.27 (W)

4. (28, 6): 4.72 (N)

5. (30, 5): 6.18 (N)



Classification Example

Standard k-NN (k=5, Majority Vote)

Votes:

- W: (25,3), (20,2) → **2 votes**
- N: (22,4), (28,6), (30,5) → **3 votes**

Prediction: Don't Water (N)

Issue: The farthest neighbors (28,6) and (30,5) skew the result, even though they're less relevant.



Classification Example

Weighted k-NN (k=5, Inverse Distance Weights)

Step 1: Assign Weights

$$\text{Weight} = \frac{1}{\text{Distance}}$$

1. (25, 3): $\frac{1}{1.12} \approx 0.89$ (W)
2. (22, 4): $\frac{1}{2.06} \approx 0.49$ (N)
3. (20, 2): $\frac{1}{4.27} \approx 0.23$ (W)
4. (28, 6): $\frac{1}{4.72} \approx 0.21$ (N)
5. (30, 5): $\frac{1}{6.18} \approx 0.16$ (N)

Why It's Better:

- The very close neighbor (25,3 → W) dominates with high weight (0.89).
- Far neighbors (e.g., 30,5 → N) have minimal impact (weight = 0.16).

Step 2: Weighted Votes

- **W:** $0.89 + 0.23 = 1.12$
- **N:** $0.49 + 0.21 + 0.16 = 0.86$

Prediction: Water (W) ($1.12 > 0.86$)



KNN Algorithm: Choosing k value

Avoid the following:

- **Small k (e.g., $k=1$):**
 - Highly sensitive to noise (overfitting).
 - Decision boundary is very flexible.
- **Large k (e.g., $k=20$):**
 - Smoother decision boundary (underfitting).
 - May misclassify if neighbors are too far.



KNN Algorithm: Choosing k value

- Choosing the right k (number of neighbors) is crucial for k-NN performance. Here's how to do it systematically:
 1. The Gold Standard: Cross-Validation
 2. Start with Rule-of-Thumb Values



KNN Algorithm: Choosing k value

1. The Gold Standard: Cross-Validation

What It Does

- Tests multiple k values on different data splits to find the most robust one.
- Prevents overfitting/underfitting by evaluating generalization.

Steps

1. **Split data** into N folds (typically 5 or 10).
2. **For each k candidate** (e.g., 1, 3, 5, 10, etc.):
 - Train on $N-1$ folds, validate on the held-out fold.
 - Repeat for all folds and average accuracy.
3. **Pick k with the highest average validation accuracy.**



KNN Algorithm: Choosing k value

1. The Gold Standard: Cross-Validation

Example

k	Fold 1 Acc.	Fold 2 Acc.	Fold 3 Acc.	Avg. Acc.	
1	85%	80%	75%	80%	
3	90%	95%	85%	90%	← Best k
5	85%	80%	85%	83%	
10	75%	70%	80%	75%	

Winner: $k=3$ (highest average accuracy).



KNN Algorithm: Choosing k value

2. Rule of Thumb: Square Root of n

- For a quick estimate, try:

$$k \approx \sqrt{n}, \text{ where } n = \text{number of training samples}$$

- **Example:** If $n=100$, test $k \approx 10$ (then refine with cross-validation).
- **Use odd k for binary classification** to avoid ties (e.g., 3, 5, 7).
- **For multi-class**, ensure k isn't a multiple of the number of classes.



Advantages of k-NN

1. Simple and Easy to Implement

- **No complex training phase:** Stores training data as-is ("lazy learner").
- **Works well with small datasets:** No model fitting required.

2. Works for Both Classification and Regression

- **Classification:** Predicts labels via majority voting.
- **Regression:** Predicts values using (weighted) neighbor averages.

3. Adapts to New Data Easily

- Adding new data points doesn't require retraining—just append to the dataset.

4. Few Hyperparameters

- Only need to tune:
 - **k** (number of neighbors).
 - Distance metric (Euclidean, Manhattan, etc.).



Disadvantages of k-NN

1. Computationally Expensive

- Must compute distances between test points and **all** training points → Slow for large datasets.
- Prediction time increases with data size (unlike parametric models).

2. Sensitive to Irrelevant Features

- If features are not scaled, high-magnitude features (e.g., salary) dominate distance calculations over smaller ones (e.g., age).
- **Solution:** Normalize/standardize features.

3. Struggles with High-Dimensional Data (Curse of Dimensionality)

- In high dimensions, distances between points become less meaningful → Poor performance.
- **Solution:** Use dimensionality reduction (PCA, feature selection).



Disadvantages of k-NN

4. Choice of k and Distance Metric Affects Performance

- **Small k :** Noisy data leads to overfitting.
- **Large k :** May oversmooth and lose fine-grained patterns.
- **Solution:** Use cross-validation to pick the best k .

5. Imbalanced Data Issues

- If one class dominates, majority voting can be biased.
- **Solution:** Use weighted k-NN or resampling (SMOTE).

6. Not Memory Efficient

- Stores the entire training dataset → High memory usage for big data.



When to Use/Avoid k-NN?

When to Use k-NN?

- ✓ Small to medium-sized datasets.
- ✓ Low-dimensional data (or after dimensionality reduction).
- ✓ When interpretability is important (e.g., "This patient is similar to these 5 cases").
- ✓ Non-linear decision boundaries needed.

When to Avoid k-NN?

- ✗ Very large datasets (slow predictions).
- ✗ High-dimensional data (e.g., text, images).
- ✗ Real-time applications (due to slow inference).



Any Question?

